

**Zadanie nr 3 – Klasyfikacja obrazów kart
do gry
przy użyciu konwolucyjnych sieci
neuronowych**

Techniki Inteligentnej Analizy Danych

Nikodem Nowak, 251598 Bartosz Kołaciński, 251554

2026-05-06

Spis treści

1	Cel zadania	1
2	Wstęp teoretyczny	1
2.1	Konwolucyjne sieci neuronowe	1
2.2	Uczenie z transferem (transfer learning)	2
2.3	Trening dwufazowy	2
2.4	Augmentacja danych	3
2.5	Miary jakości	3
3	Opis implementacji	3
3.1	Organizacja kodu	3
3.2	Procedura treningu dwufazowego	3
3.3	Pętla eksperymentów	4
3.4	Ewaluacja i artefakty	4
4	Eksperymenty i wyniki	4
4.1	Środowisko obliczeniowe	4
4.2	Dataset	5
4.3	Konfiguracja eksperymentów	5
4.4	Wyniki – skuteczność modeli (accuracy)	5
4.5	Wyniki – macro F1	6
4.6	Wyniki – AUC <i>macro</i>	6
4.7	Wpływ podziału train/test na metryki	6
4.8	Macierz pomyłek najlepszego modelu	6
4.9	Krzywe ROC najlepszego modelu	7
4.10	Krzywe uczenia	8
5	Wnioski	9

1 Cel zadania

Celem zadania było zaprojektowanie, wytrenowanie oraz porównanie skuteczności kilku architektur konwolucyjnych sieci neuronowych w problemie klasyfikacji obrazów. Zgodnie z wymogami zadania na ocenę 5 (treść instrukcji [9]) przygotowano następujący zakres prac:

- porównanie pięciu różnych modeli sieci konwolucyjnych: MobileNetV2 [10], ResNet50 [4], EfficientNetB0 [14], InceptionV3 [13] oraz VGG16 [11];
- analiza wpływu proporcji podziału zbioru na uczący i testowy: 50/50, 60/40, 70/30, 80/20 oraz 90/10;
- ewaluacja jakości klasyfikacji za pomocą macierzy pomyłek oraz miar *accuracy*, *precision*, *recall* i *F1* obliczanych w wariantach *macro*;
- przedstawienie krzywej ROC oraz wartości AUC w wariancie *one-vs-rest* (OvR), z wykorzystaniem modułu `scikit-learn` [7].

Łącznie wykonano $5 \cdot 5 = 25$ niezależnych eksperymentów treningowych.

2 Wstęp teoretyczny

2.1 Konwolucyjne sieci neuronowe

Konwolucyjne sieci neuronowe (CNN) stanowią dominujące podejście w klasyfikacji obrazów od momentu publikacji architektury AlexNet w 2012 r. Ich kluczową cechą jest hierarchiczny ekstraktor cech: wczesne warstwy konwolucyjne uczą się rozpoznawać proste wzorce (krawędzie, gradienty kolorów), warstwy środkowe tekstury i części obiektów, a najgłębsze całe obiekty oraz ich kompozycje [11].

Każda z wykorzystanych w pracy architektur reprezentuje odrębny pomysł na efektywną budowę takiego ekstraktora:

- **VGG16** [11] – klasyczna, "płaska" architektura zbudowana z powtarzających się bloków **Conv-Conv-Pool**; około 138 mln parametrów (z czego znaczna większość przypada na warstwy w pełni połączone, które w naszym podejściu zostały zastąpione własną głową);
- **InceptionV3** [13] – wykorzystuje moduły Inception, w których wyniki konwolucji 1×1 , 3×3 i 5×5 działających równolegle są łączone, co umożliwia analizę cech w różnych skalach przestrzennych;
- **ResNet50** [4] – wprowadza połączenia rezydualne (*skip connections*) postaci $f(x) + x$, dzięki którym możliwe stało się trenowanie sieci o dziesiątkach warstw bez problemu zanikającego gradientu;
- **MobileNetV2** [10] – zoptymalizowana pod urządzenia mobilne (3,5 mln parametrów); kluczową techniką są *depthwise separable convolutions*, rozkładające standardową konwolucję na dwie tańsze obliczeniowo operacje;

- **EfficientNetB0** [14] – wprowadza *compound scaling*, czyli matematycznie wyznaczone optymalne proporcje skalowania głębokości, szerokości i rozdzielczości wejścia.

2.2 Uczenie z transferem (transfer learning)

W zadaniu klasyfikacji 14 typów kart dysponowaliśmy zbiorem o stosunkowo niewielkiej liczebności (8154 obrazy). Trening sieci od podstaw (z losowo zainicjalizowanymi wagami) na takim zbiorze prowadziłby do dramatycznego niedouczenia, gdyż współczesne CNN posiadają dziesiątki milionów parametrów. Rozwiązaniem stosowanym powszechnie w tej sytuacji jest **transfer learning** [15], polegający na wykorzystaniu wag wytrenowanych wcześniej na dużym, ogólnym zbiorze danych – tutaj ImageNet [2] (1,2 mln obrazów, 1000 klas).

W naszym pipeline każdy z wymienionych modeli był ładowany z wagami z ImageNetu, a następnie modyfikowany w następujący sposób:

1. odcięto oryginalną warstwę klasyfikującą (parametr `include_top=False`);
2. zastosowano *global average pooling*, redukując mapy cech do pojedynczego wektora;
3. dodano warstwę **Dropout** [12] z prawdopodobieństwem 0,5 (regularyzacja przeciw przeuczeniu);
4. dodano warstwę **Dense** z 14 neuronami i aktywacją *softmax* (jedna na każdy typ karty).

2.3 Trening dwufazowy

Ze względu na różnicę dziedzinową między ImageNetem (zdjęcia obiektów codziennych) a obrazami kart (schematyczne, płaskie wzory), zastosowano dwufazowy schemat treningu:

1. **Faza 1** – "zamrożony" backbone (`trainable=False`); trenowane są wyłącznie wagi własnej głowy klasyfikującej. Optymalizator Adam [6] z $\eta = 10^{-3}$, do 8 epok;
2. **Faza 2** – odmrożenie 1/3 najwyższych warstw backbone'u (warstwy bliżej wyjścia są bardziej specyficzne dla domeny i wymagają dostrojenia, niższe pozostają niezmiennione). Warstwy **BatchNormalization** [5] celowo pozostawiono w trybie inference, aby uniknąć destabilizacji statystyk populacji obliczonych na ImageNecie. Optymalizator Adam z $\eta = 10^{-5}$ (wartość 100-krotnie mniejsza niż w fazie 1, gdyż modyfikujemy wagi już dobrze ustawione), do 12 epok.

W obu fazach zastosowano **EarlyStopping** monitorujący `val_loss` z parametrem `patience=5` oraz `restore_best_weights=True` (po przerwaniu treningu przywracane są wagi z epoki o najmniejszej stracie walidacyjnej).

2.4 Augmentacja danych

W trakcie treningu (wyłącznie na zbiorze uczącym) zastosowano losowe modyfikacje obrazów: rotację $\pm 10,8^\circ$, zoom $\pm 8\%$, translację $\pm 5\%$, jasność $\pm 15\%$ oraz kontrast $\pm 15\%$. Świadomie zrezygnowano z odbicia poziomego (*horizontal flip*), gdyż karty są obrazami zorientowanymi – np. figura króla po odbiciu nie jest już figurą króla.

2.5 Miary jakości

Macierz pomyłek. Tablica 14×14 , w której element $CM[i][j]$ oznacza liczbę obrazów klasy i zaklasyfikowanych jako j . Wartości na przekątnej odpowiadają poprawnym klasyfikacjom.

Accuracy, precision, recall, F1. Standardowe miary obliczone na podstawie liczby trafień (TP, FP, FN, TN); dla problemu wieloklasowego raportujemy uśrednienie *macro* (równa waga każdej klasy), które jest wrażliwe na klasy mniej liczne.

ROC i AUC. Miary niezależne od progu decyzyjnego [3]. Dla zadania wieloklasowego stosuje się wariant *one-vs-rest*: dla każdej klasy traktuje się ją jako "pozytywną", a pozostałe 13 jako "negatywne", po czym uśrednia AUC po klasach. Wartości AUC bliskie 1,0 oznaczają, że model trafnie szereguje obrazy nawet jeżeli ostateczna predykcja *top-1* nie zawsze jest poprawna.

3 Opis implementacji

3.1 Organizacja kodu

Implementacja zawarta jest w pojedynczym pliku `notebook.py` w środowisku `marimo`, które zapisuje notebook jako zwykły moduł Pythona z funkcjami opatrzonymi dekoratorem `@app.cell`. Kolejność wykonania ustalana jest automatycznie na podstawie zależności między zwracanymi zmiennymi, co umożliwia uruchamianie pliku zarówno w trybie interaktywnym (`marimo edit`), jak i wsadowym (`python notebook.py`).

Komórki podzielone są tematycznie: konfiguracja i wykrywanie GPU, wczytanie i wstępna obróbka danych z `cards.csv`, definicja funkcji generującej stratyfikowane podziały, budowa potoku `tf.data` z augmentacją, fabryka modeli wykorzystująca `keras.applications`, procedura treningu, funkcja ewaluacji oraz komórki generujące tabele i wykresy zbiorcze.

3.2 Procedura treningu dwufazowego

Kluczowym elementem implementacji jest funkcja realizująca dwufazowy schemat opisany w sekcji 2.3. W pierwszej fazie trenowane są wyłącznie wagi własnej głowy klasyfikującej – backbone pozostaje zamrożony, a optymalizator korzysta z większego learning rate. Po zakończeniu pierwszej fazy odmrażana

jest górna 1/3 warstw backbone'u (z wyłączeniem warstw normalizacji wsadowej [5], które pozostają w trybie inference, aby uniknąć destabilizacji statystyk populacji wyznaczonych pierwotnie na ImageNecie), po czym następuje krótki etap fine-tuningu z learning rate zmniejszonym o dwa rzędy wielkości. Historia obu faz jest scalana i wykorzystywana do wygenerowania spójnego wykresu krzywych uczenia.

3.3 Pętla eksperymentów

Główna pętla iteruje po wszystkich parach (model, podział) i zapisuje wyniki do pliku `results/metrics.csv` po każdym ukończonym eksperymencie. Mechanizm wznowienia sprawdza obecność wpisów w pliku CSV i pomija pary już przetworzone, co umożliwia ponowne uruchomienie pipeline'u po ewentualnej awarii bez utraty wcześniejszych wyników. Po każdym uruchomieniu wywołwana jest funkcja `tf.keras.backend.clear_session()` zwalniająca pamięć GPU.

3.4 Ewaluacja i artefakty

Funkcja `evaluate` oblicza wszystkie metryki za pomocą `scikit-learn` [7]: macierz pomyłek, `accuracy`, `precision`, `recall` i `F1` (wariant *macro*) oraz `AUC` w wariancie *one-vs-rest*. Surowe predykcje (prawdopodobieństwa) i etykiety zapisywane są do plików `numpy`, dzięki czemu wykresy ROC oraz dodatkowe analizy mogą zostać wygenerowane bez ponownego uruchamiania treningu. Wszystkie wykresy zamieszczone w niniejszym sprawozdaniu pochodzą bezpośrednio z artefaktów wygenerowanych przez ten pipeline.

4 Eksperymenty i wyniki

4.1 Środowisko obliczeniowe

Eksperymenty przeprowadzono w systemie WSL2 (Ubuntu 22.04) na komputerze wyposażonym w kartę NVIDIA GeForce RTX 5080 (16 GB VRAM, architektura Blackwell, sm_120). Implementacja w języku Python 3.12 z wykorzystaniem następujących bibliotek:

- TensorFlow 2.21 z dodatkiem `[and-cuda]` [1] – definicja i trening modeli (CUDA 12.x, cuDNN 9.x);
- Keras Applications – gotowe architektury sieci wraz z wagami z ImageNetu;
- scikit-learn 1.8 [7] – obliczanie metryk (`confusion_matrix`, `classification_report`, `roc_curve`, `roc_auc_score`);
- matplotlib oraz seaborn – wykresy, w szczególności heatmapy macierzy pomyłek;
- pandas oraz numpy – przetwarzanie danych tabelarycznych i operacje numeryczne;

- **marimo** – środowisko notebookowe (alternatywa dla Jupytera, plik źródłowy w postaci czystego pliku `.py`).

4.2 Dataset

Wykorzystano zbiór *Cards Image Dataset-Classification* [8] pobrany z platformy Kaggle za pomocą narzędzia **kaggle CLI**. Po odfiltrowaniu jednego błędnego wpisu w pliku CSV, zbiór zawiera 8154 obrazy w formacie 224×224 RGB.

Pierwotne zadanie definiuje 53 klasy (52 karty z pełnej talii oraz joker). Po wstępnych eksperymentach stwierdzono, że klasyfikacja na tym poziomie szczegółowości daje wyniki rzędu 60–72% accuracy, co wynika z ograniczonej liczby obrazów na klasę (około 150). W celu uzyskania bardziej miarodajnych wyników i lepszej analizy zdecydowano się na klasyfikację 14 typów kart (*ace*, 2–10, *jack*, *queen*, *king* oraz *joker*). Dzięki tej redukcji każda klasa zawiera średnio około 580 obrazów.

Autor zbioru udostępnia gotowy podział `train/valid/test`, jednak ze względu na wymóg analizy różnych proporcji w zadaniu, scalono cały zbiór i wykonano własne podziały z użyciem funkcji `train_test_split` z parametrem `stratify`, gwarantującym zachowanie proporcji klas w obu zbiorach.

4.3 Konfiguracja eksperymentów

Główne hiperparametry zostały zebrane w jednym miejscu kodu (Tab. 1).

Tabela 1: Konfiguracja eksperymentów

Parametr	Wartość
Rozmiar wejścia	$224 \times 224 \times 3$
Batch size	32
Optymalizator	Adam
Faza 1: liczba epok / lr	8 / 10^{-3}
Faza 2: liczba epok / lr	12 / 10^{-5}
Frakcja odmrożona w fazie 2	1/3 najwyższych warstw
Dropout	0,5
EarlyStopping (<i>patience</i>)	5
Funkcja straty	<code>sparse_categorical_crossentropy</code>
Splity train/test	0,5; 0,6; 0,7; 0,8; 0,9
Seed losowy	42

4.4 Wyniki – skuteczność modeli (accuracy)

Wartości accuracy dla wszystkich 25 eksperymentów zebrane są w Tab. 2.

Najlepszy wynik osiągnęła architektura **VGG16** przy podziale 90/10 z accuracy **0,852** oraz AUC *macro* równym **0,991**.

Tabela 2: Accuracy modeli przy różnych podziałach train/test

Model	0,5	0,6	0,7	0,8	0,9
MobileNetV2	0,691	0,728	0,742	0,755	0,761
ResNet50	0,778	0,806	0,805	0,846	0,842
EfficientNetB0	0,722	0,746	0,759	0,781	0,771
InceptionV3	0,659	0,676	0,702	0,721	0,754
VGG16	0,797	0,813	0,822	0,841	0,852

4.5 Wyniki – macro F1

Wartości macro F1 dla wszystkich 25 eksperymentów zebrane są w Tab. 3.

Tabela 3: Macro F1 modeli przy różnych podziałach train/test

Model	0,5	0,6	0,7	0,8	0,9
MobileNetV2	0,694	0,728	0,743	0,762	0,756
ResNet50	0,779	0,808	0,809	0,847	0,843
EfficientNetB0	0,728	0,755	0,768	0,788	0,779
InceptionV3	0,663	0,678	0,708	0,727	0,756
VGG16	0,802	0,816	0,828	0,849	0,858

4.6 Wyniki – AUC *macro*

Wartości AUC *macro* (one-vs-rest) dla wszystkich 25 eksperymentów zebrane są w Tab. 4.

Tabela 4: AUC *macro* (one-vs-rest) modeli przy różnych podziałach train/test

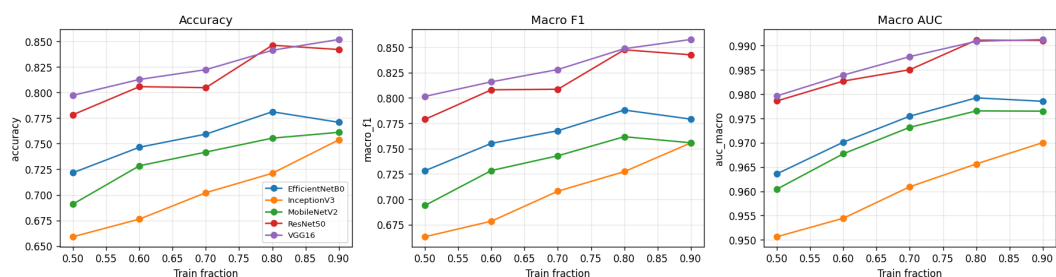
Model	0,5	0,6	0,7	0,8	0,9
MobileNetV2	0,960	0,968	0,973	0,977	0,977
ResNet50	0,979	0,983	0,985	0,991	0,991
EfficientNetB0	0,964	0,970	0,975	0,979	0,979
InceptionV3	0,951	0,954	0,961	0,966	0,970
VGG16	0,980	0,984	0,988	0,991	0,991

4.7 Wpływ podziału train/test na metryki

Graficzną interpretację wyników z Tab. 2, 3 i 4 przedstawia Rys. 1. Dla każdej z trzech metryk widoczna jest tendencja monotonicznego wzrostu wraz ze zwiększaniem udziału zbioru uczącego.

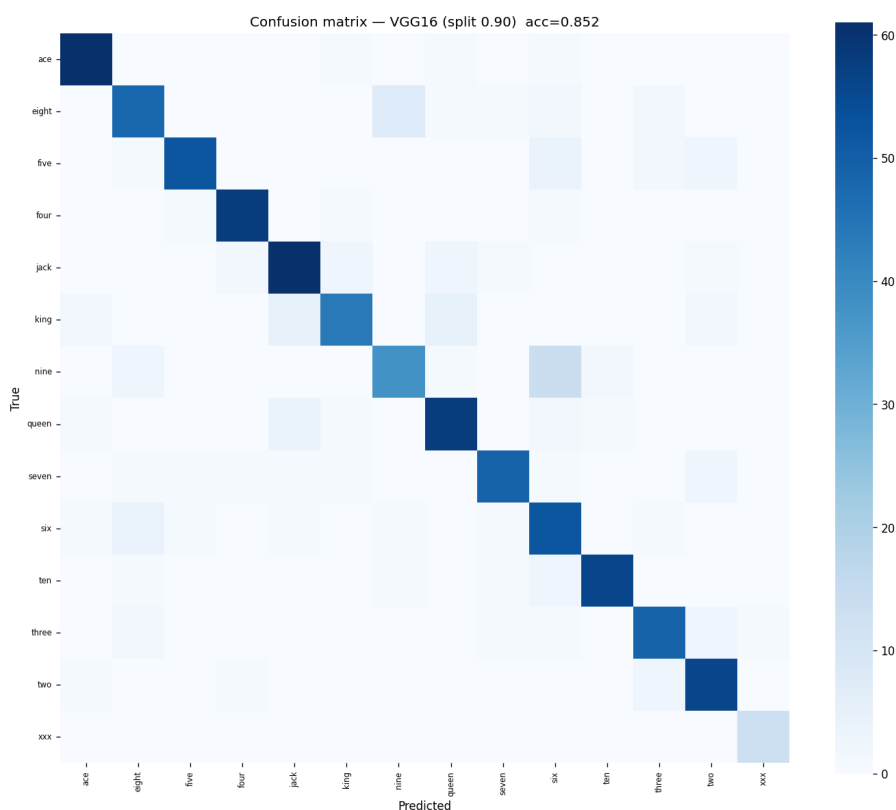
4.8 Macierz pomyłek najlepszego modelu

Macierz pomyłek dla najlepszego eksperymentu (VGG16, split 90/10) przedstawia Rys. 2. Na przekątnej widoczne są wartości znacząco większe niż poza nią,



Rysunek 1: Wpływ proporcji podziału train/test na accuracy, macro F1 i macro AUC. Każda krzywa odpowiada jednej architekturze.

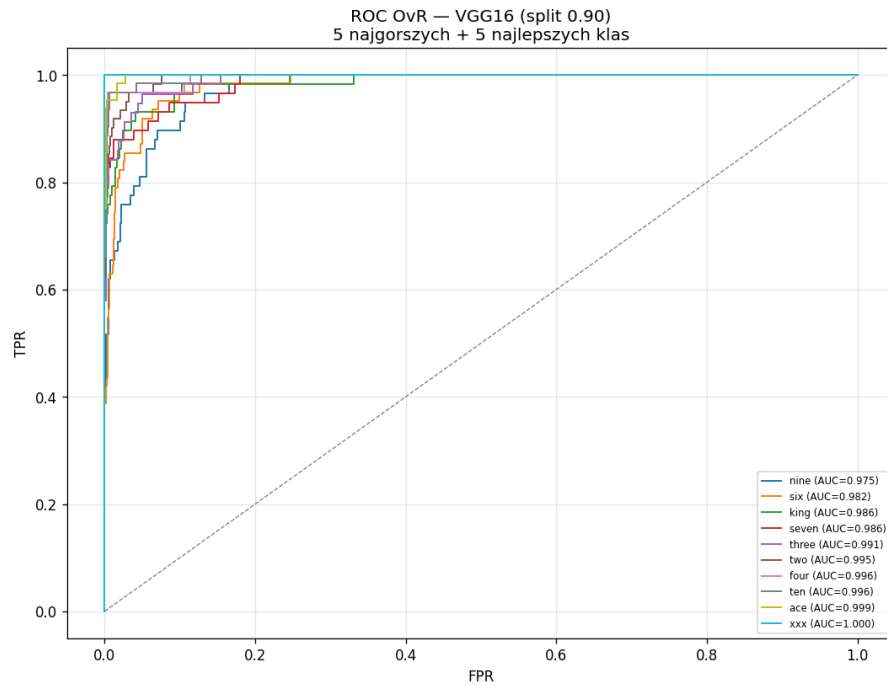
co potwierdza skuteczność klasyfikatora. Najwyraźniej zaznacza się pomyłka między kartami *six* oraz *nine*, co odpowiada symetrii obrotowej tych cyfr (po obrocie o 180° wyglądają niemal identycznie). Pozostałe pomyłki rozkładają się równomiernie i mają znacznie mniejszą intensywność.



Rysunek 2: Macierz pomyłek dla modelu VGG16 przy podziale 90/10.

4.9 Krzywe ROC najlepszego modelu

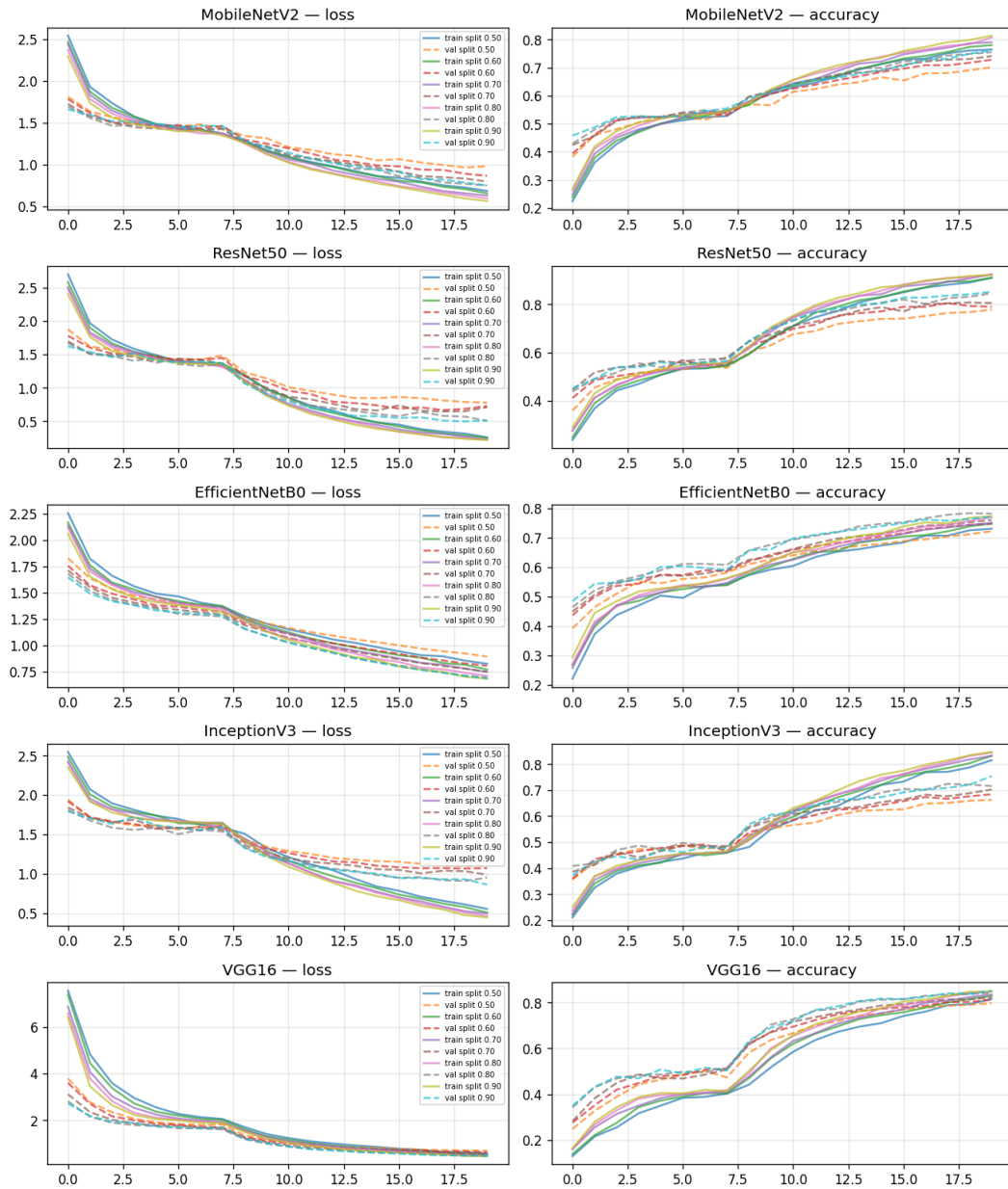
Krzywe ROC w wariantcie one-vs-rest dla pięciu najtrudniejszych i pięciu najłatwiejszych klas najlepszego modelu prezentuje Rys. 3. Wszystkie krzywe znajdują się znacząco powyżej linii losowego klasyfikatora, a wartości AUC dla większości klas przekraczają 0,97.



Rysunek 3: Krzywe ROC OvR dla VGG16 przy podziale 90/10 – pięć klas o najniższym i najwyższym AUC.

4.10 Krzywe uczenia

Przebiegi funkcji straty oraz accuracy w trakcie treningu wszystkich 25 eksperymentów przedstawia Rys. 4. Dla każdej architektury widoczna jest charakterystyczna dwuetapowość – wyraźny spadek straty na początku fazy 2 (gdy odmrożenie warstw umożliwia dostrojenie ekstraktora cech do dziedziny kart).



Rysunek 4: Krzywe uczenia (strata oraz accuracy) dla wszystkich modeli i podziałów.

5 Wnioski

1. Procedura treningu okazała się czynnikiem znacznie istotniejszym niż wybór konkretnej architektury. Trening wyłącznie głowy klasyfikującej dawał dla wszystkich modeli zbliżone wyniki (38–52% accuracy), niezależnie od liczby parametrów; dopiero wprowadzenie fazy fine-tuningu pozwoliło ujawnić rzeczywiste różnice jakościowe między architekturami.
2. Dobór augmentacji wymaga uwzględnienia specyfiki dziedziny. Powszechnie stosowane odbicie poziome (`RandomFlip`) okazało się szkodliwe ze względu na zorientowany charakter obrazów kart, podczas gdy augmen-

tacja kolorystyczna (jasność, kontrast) przyniosła wyraźną poprawę ze względu na stylistyczną różnorodność źródeł obrazów w zbiorze.

3. Granularność problemu wpływa na jakość znacznie silniej niż wybór modelu. Redukcja liczby klas z 53 do 14 typów (przy zachowaniu tego samego zbioru) podniosła accuracy o około 15 punktów procentowych, jednocześnie ujawniając czytelniejszą hierarchię modeli. Sugeruje to, że w problemach z ograniczoną liczbą próbek na klasę warto rozważyć agregację klas o ile pozwala na to definicja zadania.
4. Wartości AUC *macro* przekraczające 0,95 przy accuracy w przedziale 0,66–0,85 wskazują, że modele poprawnie szeregują klasy nawet wtedy, gdy ostateczna predykcja *top-1* jest błędna. Dla zastosowań akceptujących wskazanie kilku najbardziej prawdopodobnych klas praktyczna skuteczność jest istotnie wyższa niż sugeruje samo accuracy.
5. Nowsze architektury nie zawsze przewyższają starsze. Najwyższe wyniki osiągnęły VGG16 (2014) oraz ResNet50 (2015), podczas gdy EfficientNetB0 (2019) uzyskał wyniki średnie. Dowodzi to, że benchmarki na dużych zbiorach obrazów naturalnych nie przenoszą się wprost na dziedziny o specyficznym charakterze wizualnym.
6. Wpływ proporcji podziału jest monotoniczny w zakresie 50/50–80/20, jednak przy podziale 90/10 obserwowana jest zwiększona wariancja estymaty wynikająca z małej liczebności zbioru testowego. W badaniach wymagających większej precyzji oszacowania wskazane byłoby zastosowanie walidacji krzyżowej.
7. Wykorzystanie bardzo nowej generacji sprzętu (RTX 5080, architektura Blackwell) wymagało użycia najnowszej wersji TensorFlow oraz wiązało się z incydentalnymi problemami stabilności narzędzi w warstwie kompilacji XLA. Warto uwzględnić ten aspekt przy planowaniu podobnych eksperymentów – dojrzałość ekosystemu sprzętowego bywa równie istotna co jego wydajność.

Bibliografia

- [1] Martín Abadi et al. Tensorflow: A system for large-scale machine learning. *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 265–283, 2016.
- [2] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.
- [3] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006.

- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.
- [5] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.
- [6] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015.
- [7] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [8] Gerry Piosenka. Cards image dataset-classification. <https://www.kaggle.com/datasets/gpiosenska/cards-image-datasetclassification>, 2022. Kaggle, 8154 obrazów, 53 klasy kart.
- [9] Politechnika Łódzka. Zadanie 3 – klasyfikacja obrazów (instrukcja). Materiały kursu Techniki Inteligentnej Analizy Danych, platforma WIKAMP, 2026.
- [10] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018.
- [11] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *3rd International Conference on Learning Representations (ICLR)*, 2015.
- [12] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.
- [13] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826, 2016.
- [14] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pages 6105–6114, 2019.
- [15] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, 2014.